

informedSearchMazeFinished

March 12, 2026

1 HW3 - Informed Search Comparison

```
[1]: import numpy as np
import pandas as pd

class Node:
    def __init__(self, parent=None, position=None):
        self.parent = parent
        self.position = position
        self.g = 0
        self.h = 0
        self.f = 0

    def __eq__(self, other):
        return self.position == other.position

    def __hash__(self):
        return hash(self.position)

def InformedSearch(maze, start, end, method='AStar'):
    start_node = Node(None, start)
    end_node = Node(None, end)

    open_list = [start_node]
    closed_list = set()

    expanded_nodes = 0
    queue_size = 0

    while len(open_list) > 0:
        current_node = open_list[0]
        current_index = 0
        for index, item in enumerate(open_list):
            if item.f < current_node.f:
                current_node = item
                current_index = index
```

```

open_list.pop(current_index)
closed_list.add(current_node)

if current_node == end_node:
    path = []
    current = current_node
    while current is not None:
        path.append(current.position)
        current = current.parent
    return expanded_nodes, queue_size, path[::-1]

expanded_nodes += 1
queue_size = max(queue_size, len(open_list))

children = []
for new_position in [(0, -1), (0, 1), (-1, 0), (1, 0), (-1, -1), (-1, ↵
↵1), (1, -1), (1, 1)]:
    node_position = (
        current_node.position[0] + new_position[0],
        current_node.position[1] + new_position[1],
    )

    if (
        node_position[0] > (len(maze) - 1)
        or node_position[0] < 0
        or node_position[1] > (len(maze[len(maze) - 1]) - 1)
        or node_position[1] < 0
    ):
        continue

    if maze[node_position[0]][node_position[1]] != 0:
        continue

    children.append(Node(current_node, node_position))

for child in children:
    if child in closed_list:
        continue

    child.g = current_node.g + np.sqrt(
        np.square(child.position[0] - current_node.position[0])
        + np.square(child.position[1] - current_node.position[1])
    )
    child.h = np.sqrt(
        np.square(child.position[0] - end_node.position[0])
        + np.square(child.position[1] - end_node.position[1])
    )

```

```

        if method == 'AStar':
            child.f = child.g + child.h
        elif method == 'GBF':
            child.f = child.h
        elif method == 'UCS':
            child.f = child.g

        child_exists = False
        for open_node in open_list:
            if child == open_node and child.g >= open_node.g:
                child_exists = True
                break

        if not child_exists:
            open_list.append(child)

def pathLength(path):
    dis = 0
    for i in range(len(path) - 1):
        x1, y1 = path[i]
        x2, y2 = path[i + 1]
        dis += np.sqrt(np.square(x1 - x2) + np.square(y1 - y2))
    return dis

def mazeRunner(start, end, maze, searchMethod='AStar'):
    maze[start[0]][start[1]] = 0
    maze[end[0]][end[1]] = 0
    expanded_nodes, queue_size, path = InformedSearch(maze, start, end,
↪searchMethod)
    return expanded_nodes, queue_size, path, pathLength(path)

```

```

[2]: maze = [[0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
             [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
             [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
             [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
             [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
             [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
             [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
             [0, 0, 0, 0, 0, 0, 0, 0, 1, 0],
             [0, 0, 0, 0, 0, 0, 0, 1, 1, 0],
             [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]]

start = (0, 0)
end = (9, 9)

gbf = mazeRunner(start, end, [row[:] for row in maze], 'GBF')

```

```

ucs = mazeRunner(start, end, [row[:] for row in maze], 'UCS')
astar = mazeRunner(start, end, [row[:] for row in maze], 'AStar')

results = pd.DataFrame([
    {'Method': 'GBF', 'Expanded Nodes': gbf[0], 'Max Queue Size': gbf[1], 'Path_
↪Length': gbf[3], 'Path': gbf[2]},
    {'Method': 'UCS', 'Expanded Nodes': ucs[0], 'Max Queue Size': ucs[1], 'Path_
↪Length': ucs[3], 'Path': ucs[2]},
    {'Method': 'AStar', 'Expanded Nodes': astar[0], 'Max Queue Size': astar[1],
↪'Path Length': astar[3], 'Path': astar[2]},
])
results

```

```

[2]:   Method  Expanded Nodes  Max Queue Size  Path Length \
0    GBF                11                30    14.727922
1    UCS                104                22    13.899495
2  AStar                 54                59    13.899495

```

```

                                Path
0  [(0, 0), (1, 1), (2, 2), (3, 3), (4, 4), (5, 5...
1  [(0, 0), (0, 1), (0, 2), (1, 3), (2, 4), (3, 5...
2  [(0, 0), (1, 1), (2, 2), (3, 3), (4, 4), (5, 5...

```

1.1 (a) Paths and Optimality

GBF path: [(0,0), (1,1), (2,2), (3,3), (4,4), (5,5), (6,6), (7,7), (6,8), (7,9), (8,9), (9,9)]
 UCS path: [(0,0), (0,1), (0,2), (1,3), (2,4), (3,5), (4,6), (5,7), (6,8), (7,9), (8,9), (9,9)]
 AStar path: [(0,0), (1,1), (2,2), (3,3), (4,4), (5,5), (6,6), (6,7), (6,8), (7,9), (8,9), (9,9)]

With this maze, UCS and AStar achieve the same lower path cost (13.899495) and are optimal.

GBF gives a larger path cost 14.727922, so it is not optimal here.

This is because UCS optimizes cumulative cost $g(n)$, AStar optimizes by adding a heuristic $g(n)+h(n)$, while GBF uses only heuristic $h(n)$ and can make locally greedy but globally sub-optimal decisions.

1.2 (b) Advantage of AStar over GBF and UCS

- Over GBF: AStar keeps optimality guarantees (with admissible heuristic), GBF does not.
- Over UCS: AStar usually expands fewer nodes by using heuristic direction, so it is often more efficient while still optimal.

1.3 (c) Advantage of GBF over AStar and UCS

- GBF is often simpler and faster to run in practice because it only uses $h(n)$ and aggressively moves toward the goal.
- Tradeoff: this speed can reduce solution quality since GBF is not guaranteed to return an optimal path.